

Dynamic Prompt Builder

Objective

The objective of this task was to understand how user input can be transformed into a structured AI prompt using a frontend application and a FastAPI backend.

This project demonstrates the concept of Prompt Engineering, where raw user information is converted into a well-formatted prompt that can later be used by AI models such as ChatGPT, Gemini, Claude, or other Large Language Models (LLMs).

Technologies Used

- HTML
 - CSS
 - JavaScript
 - FastAPI
 - Python
 - JSON
-

Project Overview

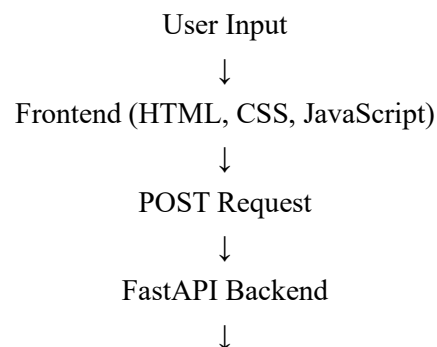
A Dynamic Prompt Builder application was developed.

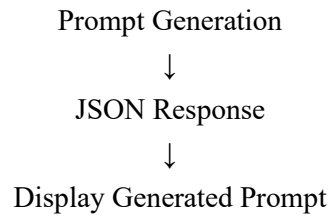
The application allows users to enter:

- Project Name
- Skills Used

After clicking the "Generate Prompt" button, the frontend sends the information to the FastAPI backend. The backend formats the data into a structured prompt and returns it to the frontend for display.

System Architecture





Frontend Implementation

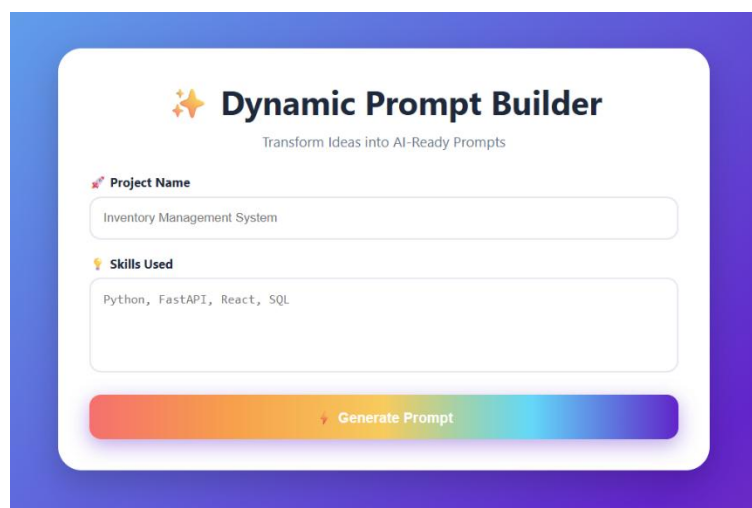
The frontend was developed using:

- HTML for structure
- CSS for styling
- JavaScript Fetch API for communication with the backend

Features:

- Modern colorful user interface
- Animated gradient background
- Responsive design
- Project Name input field
- Skills input field
- Generate Prompt button
- Generated Prompt display section

Screenshot 1: Frontend User Interface



Backend Implementation

The backend was developed using FastAPI.

The backend receives user data and dynamically creates a structured prompt.

Endpoint Used

POST /build-prompt

Screenshot 2: FastAPI Source Code (main4.py)

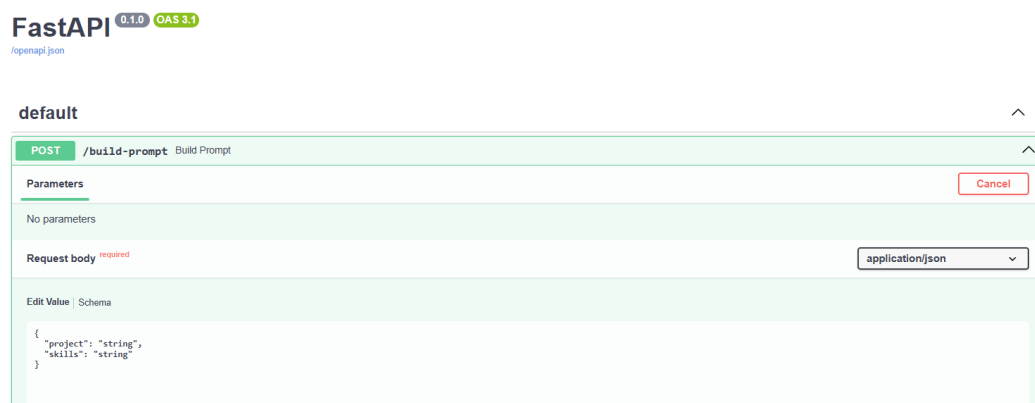
```
C: > Users > Admin > Desktop > main4.py > ...
1  from fastapi import FastAPI
2  from fastapi.middleware.cors import CORSMiddleware
3  from pydantic import BaseModel
4
5  app = FastAPI()
6
7  app.add_middleware(
8      CORSMiddleware,
9      allow_origins=["*"],
10     allow_credentials=True,
11     allow_methods=["*"],
12     allow_headers=["*"],
13 )
14
15 class PromptData(BaseModel):
16     project: str
17     skills: str
18
19 @app.post("/build-prompt")
20 def build_prompt(data: PromptData):
21     prompt = f"""
22     Generate ATS friendly bullet points.
23
24     Project:
25     {data.project}
26
27     Skills:
28     {data.skills}
29
30     Requirements:
31     - Use action verbs
32     - Keep points professional
33     - Highlight technical skills
34     - Focus on measurable impact
35     """
36
37     return {
38         "prompt": prompt
39     }
40
```

API Testing

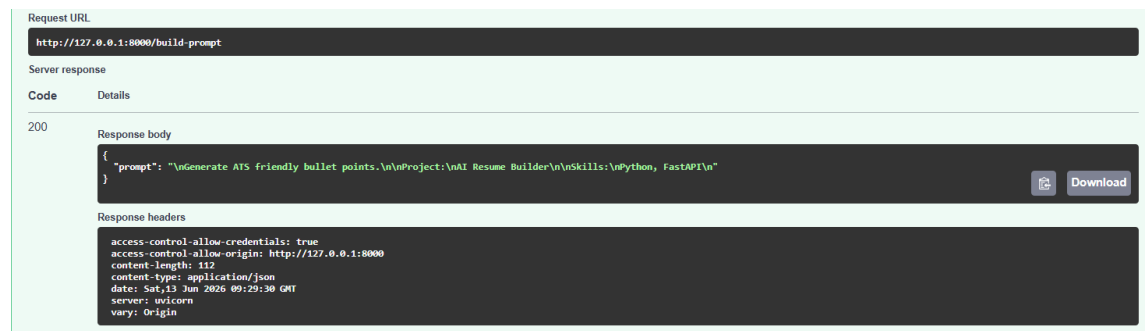
The endpoint was tested using FastAPI Swagger Documentation.

The API successfully received the Project Name and Skills and returned a formatted prompt.

Screenshot 3: FastAPI Swagger Documentation



Screenshot 4: Successful API Response



Working Process

Step 1:

User enters Project Name and Skills.

Step 2:

Frontend sends a POST request to the backend.

Step 3:

FastAPI receives the JSON data.

Step 4:

Backend dynamically generates a prompt template.

Step 5:

Prompt is returned as a JSON response.

Step 6:

Frontend displays the generated prompt.

Sample Input

Project Name:

AI Resume Builder

Skills:

Python, FastAPI

Generated Prompt

Generate ATS friendly bullet points.

Project:

AI Resume Builder

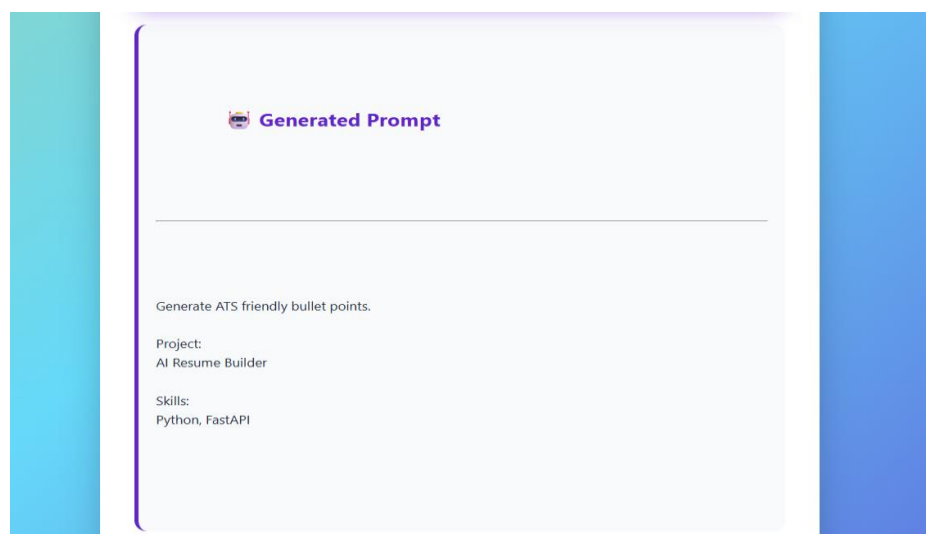
Skills:

Python, FastAPI

Requirements:

- Use action verbs
- Keep points professional
- Highlight technical skills
- Focus on measurable impact

Screenshot 5: Generated Prompt Displayed on Frontend



Learning Outcomes

Through this project, the following concepts were learned:

- Client-Server Architecture
 - FastAPI API Development
 - JSON Request and Response Handling
 - JavaScript Fetch API
 - Dynamic Data Processing
 - Prompt Engineering
 - Prompt Templates
 - Frontend and Backend Integration
 - API Testing using Swagger
-

Importance of Prompt Engineering

AI models perform better when provided with structured prompts.

Instead of sending raw user input directly to an AI model, applications first convert the data into a well-defined prompt.

This project demonstrates the first step in building AI-powered applications by generating prompts dynamically from user input.

Conclusion

The Dynamic Prompt Builder successfully converts user input into a structured AI-ready prompt. The project demonstrates how frontend applications communicate with backend services and how prompt engineering can be implemented before integrating real AI models. This serves as a foundation for future AI-powered applications such as Resume Builders, Chatbots, and Content Generation Systems.